

CatLin Studio



Evaluación Final — JavaScript



Aplicación Web Interactiva · Simulador de Reservas

Desarrollo Frontend · 2026

1. Proyecto / Contexto

¿Qué es CatLin Studio?

Estudio de belleza y bienestar con servicios de manicura, diseño de cejas, extensiones de pestañas y masajes relajantes. El sitio fue transformado de una página estática a una **aplicación web interactiva** que simula el proceso completo de reserva de turnos.

Circuito de negocio simulado

1. **Explorar** — Catálogo de servicios con filtros por categoría
2. **Seleccionar** — Agregar servicios a la reserva con un clic
3. **Personalizar** — Elegir fecha y horario disponible
4. **Registrarse** — Completar datos personales (nombre, email, teléfono)
5. **Confirmar** — Revisar resumen y confirmar la reserva
6. **Recibir** — Obtener un número de reserva único como comprobante

Tecnologías aplicadas

HTML5 semántico

CSS3 / SASS (7-1)

Bootstrap 5

JavaScript (ES6+)

SweetAlert2

AOS

JSON + Fetch

Git / GitHub

Vercel

Conceptos clave de JavaScript aplicados

Concepto	Implementación
Variables (let/const)	Estado de reserva, configuración AOS, selección de servicios
Arrays y objetos	Servicios, testimonios, carrito de reserva, subservicios
Métodos funcionales	filter(), find(), map(), reduce() para filtrar y calcular totales
Manipulación del DOM	createElement, innerHTML, addEventListener, classList, querySelector
Eventos	click, submit, input, scroll — con delegación de eventos
Fetch asincrónico	Carga de services.json y testimonials.json con promesas
SweetAlert2 (librería)	Modales de confirmación, notificaciones, sin alert/confirm/prompt

2. Código — Archivos JSON

data/services.json

```
[
  {
    "id": 1,
    "name": "Manicura Clásica",
    "category": "manicura",
    "description": "Manicura tradicional con esmaltado...",
    "price": 25, "duration": 45,
    "image": "https://images.unsplash.com/...",
    "icon": "fa-hand-sparkles",
    "subservices": [
      { "name": "Manicura clásica", "price": 25 },
      { "name": "Semipermanente", "price": 35 }
    ]
  },
  { "id": 2, "name": "Diseño de Cejas", "category": "cejas", ... },
  { "id": 3, "name": "Extensiones de Pestañas", "category": "pestanas", ... },
  { "id": 4, "name": "Masajes Relajantes", "category": "masajes", ... },
  { "id": 5, "name": "Paquete Básico", "category": "paquetes", ... },
  { "id": 6, "name": "Paquete Premium", "category": "paquetes", ... },
  { "id": 7, "name": "Paquete Deluxe", "category": "paquetes", ... }
]
```

data/testimonials.json

```
[
  { "id": 1, "name": "María González", "rating": 5,
    "text": "El mejor servicio de manicura que he probado..." },
  { "id": 2, "name": "Ana Martínez", "rating": 5,
    "text": "Las extensiones de pestañas quedaron perfectas..." },
  { "id": 3, "name": "Laura Sánchez", "rating": 5,
    "text": "Los masajes son increíbles..." }
]
```

Ambos archivos se cargan mediante `fetch()` asíncronico. Los datos se renderizan en el DOM sin bloquear la interfaz.

3. Código — HTML5 (estructura)

pagina3.html — Simulador de reserva

```
<section class="booking-page">
  <div class="container">
    <div class="row">
      <div class="col-lg-8">
        <!-- PASO 1: Selección de servicios -->
        <div class="booking-step">
          <span class="step-number">1</span>
          <h3>Elegí tus servicios</h3>
          <div class="row" id="bookingServices"></div>
        </div>
        <!-- PASO 2: Datos y horario -->
        <div class="booking-step">
          <span class="step-number">2</span>
          <h3>Tus datos y horario</h3>
          <form id="bookingForm">
            <!-- nombre, email, teléfono, fecha, hora -->
          </form>
        </div>
      </div>
      <!-- SIDEBAR: Resumen de reserva -->
      <div class="col-lg-4">
        <div class="booking-sidebar" id="bookingSummary">
          <!-- Total + lista de servicios seleccionados -->
        </div>
      </div>
    </div>
    <button id="confirmBookingBtn">Confirmar Reserva</button>
  </div>
</section>
```

pagina1.html — Catálogo con filtros

```
<!-- Filtros por categoría (Bootstrap btn-group) -->
<div class="btn-group filter-buttons">
  <button class="filter-btn active" data-filter="todos">Todos</button>
  <button class="filter-btn" data-filter="manicura">Manicura</button>
  <button class="filter-btn" data-filter="cejas">Cejas</button>
  <button class="filter-btn" data-filter="pestanas">Pestañas</button>
  <button class="filter-btn" data-filter="masajes">Masajes</button>
  <button class="filter-btn" data-filter="paquetes">Paquetes</button>
</div>

<!-- Contenedor renderizado dinámicamente por JS -->
<div class="row g-4" id="allServicesContainer"></div>
```

Todos los HTML usan etiquetas semánticas (<header>, <main>, <section>, <footer>, <nav>), atributos de accesibilidad (aria-label, alt) y meta tags para SEO.

4. Código — JavaScript

Fetch de datos JSON

```
function fetchJSON(url) {
  return fetch(url).then(response => {
    if (!response.ok) throw new Error('Error al cargar datos');
    return response.json();
  });
}

// Uso: cargar servicios
fetchJSON('data/services.json')
  .then(services => renderServices(services, container, true))
  .catch(() => { /* fallback visual */ });
```

Renderizado dinámico con el DOM

```
function buildServiceCard(service, showAddButton) {
  const col = document.createElement('div');
  col.className = 'col-md-6 col-lg-4';
  col.innerHTML = `
    <div class="service-card h-100">
      
      <div class="card-body">
        <h3>${service.name}</h3>
        <p>${service.description}</p>
        <span class="badge">${service.price}</span>
        ${showAddButton ? '<button>Agregar</button>' : ''}
      </div>
    </div>`;
  return col;
}
```

Métodos funcionales de arrays

```
// Filtrar por categoría
const filtered = services.filter(s => s.category === categoria);

// Buscar un servicio
const service = services.find(s => s.id === id);

// Calcular total de la reserva
const total = bookingState.selectedServices
  .reduce((acc, s) => acc + s.price, 0);
```

Delegación de eventos

```
// Un solo listener para todos los botones "Agregar"
bookingContainer.addEventListener('click', (e) => {
  const addBtn = e.target.closest('.btn-add-service');
  if (!addBtn) return;
```

```
// lógica de agregado al carrito...  
});
```

SweetAlert2 (sin alert/confirm)

```
// Confirmación de reserva  
Swal.fire({  
  title: '¿Confirmar tu reserva?',  
  html: `<div>...resumen...</div>`,  
  icon: 'question',  
  showCancelButton: true,  
  confirmButtonText: 'Confirmar Reserva',  
  cancelButtonText: 'Cancelar'  
}).then((result) => {  
  if (result.isConfirmed) {  
    Swal.fire({  
      title: '¡Reserva Confirmada!',  
      html: `<div>...detalle + nº de reserva...</div>`,  
      icon: 'success'  
    });  
  }  
});
```

5. Código — SASS (Arquitectura 7-1)

Estructura de archivos

```
scss/  
├─ main.scss  
├─ abstracts/  
│   ├── _variables.scss    // Colores, fuentes, breakpoints  
│   ├── _mixins.scss       // respond-to(), transition()  
│   └── _placeholders.scss // %section-padding  
├─ base/  
│   ├── _reset.scss  
│   └── _typography.scss  
├─ components/  
│   ├── _nav.scss  
│   ├── _card.scss  
│   └── _button.scss  
├─ layout/  
│   ├── _header.scss  
│   ├── _footer.scss  
│   └── _grid.scss  
└─ pages/  
    ├── _home.scss  
    ├── _services.scss  
    ├── _about.scss  
    ├── _contact.scss  
    └── _booking.scss      // NUEVO: estilos del simulador
```

Variables (abstracts/_variables.scss)

```
$color-primary:    #e8b4b8;  
$color-primary-dark: #d4919b;  
$color-secondary:  #f5e6e8;  
$font-primary:     'Poppins', sans-serif;  
$font-secondary:   'Playfair Display', serif;  
$bp-mobile:        576px;  
$bp-tablet:        768px;  
$spacing-xl:       3rem;  
$shadow-md:        0 4px 8px rgba(0,0,0,0.15);
```

Mixins (abstracts/_mixins.scss)

```
@mixin respond-to($breakpoint) {  
  @if $breakpoint == tablet {  
    @media (max-width: $bp-tablet) { @content; }  
  } @else if $breakpoint == mobile {  
    @media (max-width: $bp-mobile) { @content; }  
  }  
}  
  
@mixin transition($props, $duration, $ease) {  
  transition-property: $props;  
  transition-duration: $duration;  
}
```

```
    transition-timing-function: $ease;  
  }
```

Estilos del simulador (pages/_booking.scss)

```
.booking-step {  
  background: white;  
  border-radius: 16px;  
  padding: 2rem;  
  box-shadow: 0 2px 4px rgba(0,0,0,0.1);  
  
  .step-header {  
    display: flex;  
    align-items: center;  
    gap: 0.5rem;  
  
    .step-number {  
      width: 36px; height: 36px;  
      background: $color-primary;  
      color: white;  
      border-radius: 50%;  
      display: flex;  
      align-items: center;  
      justify-content: center;  
      font-weight: 700;  
    }  
  }  
}
```


6. Repositorio en GitHub

<https://github.com/valeriaecheverry21/CatLin-Studio.git>

Comandos Git utilizados

```
git init
git add .
git commit -m "mensaje"
git remote add origin https://github.com/valeriaecheverry21/CatLin-Studio.git
git branch -M main
git push -u origin main
```

Historial de commits (13 incrementales)

#	Hash	Mensaje
1	ce38f50	Add .gitignore to exclude node_modules
2	7536ccc	Mejora responsive: menú hamburguesa y navegación mobile
3	4243276	Mejora responsive: cards de servicios adaptadas a mobile
4	b618a62	Mejora responsive: footer centrado para dispositivos móviles
5	1db8505	Mejora responsive: hero section adaptada a mobile
6	34717ee	Mejora responsive: formulario de contacto e info adaptados a mobile
7	e21a178	Mejora responsive: página de servicios adaptada a mobile
8	6d7081d	Mejora responsive: página nosotros adaptada a mobile
9	c3e8c62	Estilos compilados: CSS generado desde SASS
10	aa7dee8	Estructura SASS completa: variables, mixins, componentes, layout y páginas
11	24c6c61	Documentos de entrega: plantilla HTML para PDF y versión para Docs
12	9dd1fd4	Documento entrega: reemplazar placeholders con código real e instrucciones
13	624220d	Simulador interactivo de reservas: JSON + Fetch, SweetAlert2, DOM events

Cada commit representa un incremento funcional que documenta el desarrollo paso a paso.

7. Demo — Sitio Publicado

<https://cat-lin-studio-58p5uca2f-valeriaecheverry21s-projects.vercel.app/>

Capturas sugeridas

#	Pantalla	Qué mostrar
1	Home (Desktop 1920px)	Hero + servicios + testimonios desde JSON
2	Servicios (Desktop)	Filtro "Manicura" activo, botones "Agregar"
3	Reserva (Desktop)	2 servicios agregados, sidebar con total
4	Modal SweetAlert2	Confirmación con n° de reserva y detalle
5	Mobile (375px)	Menú hamburguesa + formulario responsive
6	Tablet (768px)	Servicios en 2 columnas con filtros

Pasos del despliegue (Vercel)

1. Ir a **vercel.com** y hacer login con GitHub
2. "Add New Project" → Importar repositorio CatLin-Studio
3. Configurar: Framework = *Other*, Build = `npm run build-css`, Output = `.`
4. "Deploy" → Obtener URL automática
5. Cada push a GitHub redepone automáticamente

8. Reflexión Final

El desarrollo de **CatLin Studio** como aplicación web interactiva permitió integrar y aplicar todos los conceptos aprendidos durante el curso de JavaScript y desarrollo frontend. Cada concepto teórico se tradujo en una implementación concreta dentro del proyecto.

Variables y tipos de datos

Se utilizó `const` para referencias del DOM, configuraciones y funciones. `let` para el estado mutable de la reserva. Se trabajó con objetos, arrays de objetos y tipos primitivos de forma consistente.

Funciones y ámbito

Funciones con nombre descriptivo encapsulan lógica específica: `fetchJSON()`, `buildServiceCard()`, `renderServices()`, `updateBookingSummary()`. Los closures aparecen naturalmente en callbacks de eventos y promesas.

Manipulación del DOM

Creación dinámica de elementos con `createElement` e `innerHTML`. Modificación de clases con `classList`. Scroll suave con `scrollIntoView`. **Delegación de eventos** en contenedores padres para manejar clics en botones agregados dinámicamente.

Asincronismo (Fetch + Promesas)

Los datos de servicios y testimonios se cargan desde archivos `.json` mediante `fetch()` con promesas. El manejo de errores con `.catch()` muestra fallbacks visuales sin romper la experiencia.

Métodos funcionales de arrays

`filter()` para filtrar por categoría, `find()` para buscar por ID, `map()` para transformar datos en la interfaz, `reduce()` para calcular totales. Reemplazan bucles tradicionales con código declarativo.

Librerías externas

SweetAlert2 reemplazó `alert/confirm/prompt` con modales visuales consistentes con la paleta de colores del estudio. **AOS** aporta animaciones de scroll. **Bootstrap 5** proporciona el sistema de grillas y componentes base.

Buenas prácticas

- Cero código comentado o en desuso

- Nombres descriptivos de funciones y variables
- Sin `console.log`, `alert`, `confirm` ni `prompt`
- Manejo de errores con fallbacks visuales
- Código estructurado en secciones lógicas dentro de un único archivo JS

Dificultades encontradas

Migrar de un sitio estático a interactivo sin romper funcionalidad existente. Ajustar rutas relativas en SASS con `@use`. Delegación de eventos en elementos generados dinámicamente, resuelto con `e.target.closest()`.

Conclusión

CatLin Studio demuestra la aplicación práctica de todos los conceptos del curso: desde sintaxis básica hasta asincronismo, manipulación del DOM e integración de librerías. El resultado es una aplicación web funcional, responsive, con código limpio y mantenible.